📊 Day 4: Data Understanding & Schema Design

Resume Analysis

- Skills vary across domains
- Experience is in years + designation
- Education follows structured format
- Certifications add value

Job Description Analysis

- Skills have priority levels
- Experience ranges defined
- Education is filtering criteria

Data Entities

- Candidate Profile
- Job Profile
- Skill Object
- Experience Object

Purpose

Convert unstructured resumes into structured JSON for AI processing.

📊 Day 6: Job Description Parsing Documentation

🔹 Objective

To convert unstructured job descriptions into structured data for AI processing and candidate matching.

🔹 Input

- Job descriptions in `.txt` format
- Stored in: `data/job_descriptions/`

🔹 Processing Steps

1. **Text Normalization**
    - Convert text to lowercase
    - Remove special characters and symbols
    - Clean extra spaces and formatting issues
2. **Skill Extraction**

- Match keywords from predefined skill database
- Example: Python, SQL, Machine Learning

3. **Experience Extraction**
   - Use regex to detect patterns like:
     - “2 years”
     - “3+ years”
4. **Role Detection**
   - Identify job titles based on keywords:
     - Analyst, Engineer, Developer
5. **Education Detection**
   - Detect degrees like:
     - Bachelor, Master, B.Tech, M.Tech

🔹 Output

Each job description is converted into structured JSON format:

```
{
  "job_title": "data analyst",
  "skills": ["python", "sql"],
  "experience_required": 2,
  "education": "Bachelor"
}
```

🔹 Batch Processing

- Multiple job descriptions are processed automatically
- Implemented using `jd_batch_processor.py`
- Supports scalable data handling (66+ JDs)

🔹 Advantages

- Converts unstructured text into structured data
- Enables AI-based matching and scoring
- Improves recruitment automation efficiency

📊 Day 7: Metadata Standards

🔹 Candidate Metadata

- candidate_id: Unique identifier (e.g., C001)
- name: Candidate name
- email: Contact details

🔹 Job Metadata

- job_id: Unique job identifier (e.g., J001)
- job_title: Role name

🔹 System Metadata

- model_version: Version of AI model used
- timestamp: Processing time
- source: Resume or JD file source

🔹 Example Metadata

```
{
  "candidate_id": "C001",
  "job_id": "J001",
  "model_version": "v1.0",
  "timestamp": "2026-04-18T10:00:00"
}
```

📊 Section Detection Accuracy Report

Objective

Evaluate how accurately the system classifies resume sections.

Accuracy

- Structured resumes: 85–90%
- Semi-structured resumes: 70–80%
- Unstructured resumes: 60–70%

Strengths

- Works well with standard headings
- Fast and lightweight

Limitations

- Fails when headings are missing
- Struggles with tables and columns
- Depends on keyword matching

Improvements

- Use NLP models (spaCy, BERT)
- Train ML classifier
- Expand keyword database

📊 Day 9: Skill Extraction Engine

Objective

Extract and normalize skills from resumes using NLP and rule-based techniques.

Features

- Master skill dictionary
- Synonym handling (Python = Python3)
- Skill stack detection (MERN, MEAN)
- Deduplication of skills
- Confidence scoring

Approach

1. Normalize resume text
2. Match against skill dictionary
3. Count occurrences
4. Assign confidence scores
5. Remove duplicates

Output Example

```
{
  "skills": ["python", "sql"],
  "confidence": {
    "python": 0.9,
    "sql": 0.7
  }
}
```

📊 Day 10: Experience Parsing & Scoring

Objective

Extract and analyze candidate work experience from resumes.

Features

- Company name extraction
- Role detection
- Duration calculation
- Total experience computation
- Experience relevance scoring

Approach

1. Normalize resume text
2. Extract date patterns (YYYY–YYYY)
3. Identify companies and roles
4. Calculate total experience
5. Compare roles with job description

Output Example

```
{
  "companies": ["tcs"],
  "roles": ["data analyst"],
  "total_experience_years": 2
}
```

Bias Reduction & Fairness Strategy

The system incorporates fairness mechanisms to ensure unbiased candidate evaluation:

1. **Text Normalization** All resumes and job descriptions are converted into a standardized format to eliminate formatting inconsistencies.

2. **Personal Information Masking** Sensitive attributes such as names, email addresses, and phone numbers are removed to prevent bias.

3. **Balanced Scoring** Keyword-based scoring is balanced with semantic similarity to avoid over-reliance on keyword matching.

4. **Score Normalization** All scores are normalized to maintain consistency across different candidates.

5. **Bias Detection** The system flags potential bias indicators such as gender, age, or marital status references.